SplitSmart: Smart Group Expense Settler

Data Structures Project WriteUp Idea

Navaneeth A - 2025115151
Mohamed Shaqeeq - 2025115141
Aushadha Kumar - 2025115169

1. Abstract

Anyone who has been on a trip with friends knows how the money part goes. One person pays for the hotel, another pays for the cab, someone else covers dinner, and by the end nobody really knows who owes what. People then send each other a lot of small UPI payments, and some of these payments just go around in circles. SplitSmart is a group expense settler written in C that sorts this out. The group enters every expense, the program works out the net balance of each person, and then it settles everyone with as few payments as possible, never more than one less than the number of people. For this we used a graph, two heaps (a max-heap and a min-heap), a linked list and a stack, and each one has a clear job in the program.

2. Problem Statement

On most group trips, different people pay for different things at different times. One friend keeps a rough note on their phone, another says they will send the money later, and a week after the trip half of it is still pending.

The problems we noticed:

- Since different people pay, nobody has the full picture of who is ahead and who is behind.
- If everyone just pays the payer for each expense, six friends with five shared expenses can end up making around 20 separate payments.
- Many of these payments are not needed. If A pays B and B pays C, A could have paid C directly.
- A wrong amount in a chat or notes app is hard to correct, and the whole calculation has to be done again by hand.
- Small amounts get forgotten, and that leads to awkward conversations later.

UPI has made it very easy to send money, but it does not tell anyone who should pay whom. So, the real problem is not only recording the expenses. It is bringing a messy web of small debts down to as few payments as possible, and getting it right every time.

3. Objectives

- Keep all the expenses of a group in one place instead of in scattered messages and notes.
- Work out each person's net balance correctly, whether they paid more or less than their share.
- Settle the whole group with the fewest payments possible, at most one less than the number of people.
- Let the group undo a wrong entry without redoing everything.

- Use the data structures we learned in class in a way that really helps the program.

4. Proposed Solution

SplitSmart is used by a group of friends, and any one of them can run it for everyone.

Adding members. The names of everyone in the group are entered first.

Adding expenses. For each expense, the user enters a description, the amount, who paid and who shares it. The program divides the amount equally among the people sharing it and updates everyone's balance. It also records who owes whom for that expense in a graph.

Settling up. The program separates people into those who should receive money (creditors) and those who should pay (debtors). Each group goes into its own heap. The biggest debtor then pays the biggest creditor, and this is repeated until everything is settled. The output is a short list like "Charu should pay Arun Rs. 2450".

Undo and save. The last expense can be undone if it was entered by mistake. The whole trip can also be saved to a file and loaded again later.

The flow looks like this:

```
ADD EXPENSE -> update balances and debt graph -> push on history stack

UNDO -> pop last expense from the stack -> reverse balances and graph

SETTLE UP -> split people into creditors and debtors
          -> [CREDITOR HEAP]   [DEBTOR HEAP]
          -> biggest debtor pays biggest creditor
          -> repeat until both heaps are empty
          -> final payment list
```

The heaps are what make the settling step work well. They give the program the right pair of people each time without having to sort everyone again after every payment.

5. Real-World Connection

This is not only a classroom exercise. The same problem comes up wherever a group shares money: trips with friends, hostel and PG roommates sharing groceries, flatmates splitting rent and electricity bills, college fest committees, office lunches, wedding and event budgets, and group gifts. In most of these, one person pays first and the rest settle up later.

Apps like Splitwise already do something like this, and many payment apps now have a split option. We built the main logic ourselves using the data structures from class, so we know exactly how it works and where it can fall short.

If this were taken further, each line of the final payment list could open a UPI payment directly, it could become a mobile app, and it could keep many groups in one place. The data structure logic would stay the same.

6. Data Structures Used

Here is each structure we used, what it is, where it sits in our project and why we chose it.

6.1 Graph (Adjacency Matrix)

Definition. A graph is a set of nodes joined by edges. In a directed weighted graph every edge has a direction and a value. An adjacency matrix stores this in a 2D array, where the entry in row i and column j is the value of the edge going from node i to node j.

Where we used it. Every person is a node. An edge from i to j with value x means that i owes j x rupees. Each time an expense is added, the edges from every sharer to the payer are updated.

Why. The graph shows the messy picture before settling, where each person owes several others. It lets us print all the raw debts and count them, so we can show how much SplitSmart cuts down. Checking any pair takes O(1) time, and the matrix needs $O(n^2)$ space, which is fine for a group of friends.

6.2 Heaps (Priority Queue)

Definition. A max-heap is a complete binary tree in which every parent is greater than or equal to its children, so the largest element is always at the top. Insertion and removal of the maximum take O(log n). A min-heap is the same, except that every parent is smaller than or equal to its children, so the smallest element is at the top.

Where we used it. We use two heaps. The creditor heap is a max-heap, with the person who should receive the most on top. The debtor heap is a min-heap on the balance, so the person with the most negative balance, who has to pay the most, is on top. Both heaps store the person's index and compare the current balances.

Why. At every step we need the biggest creditor and the biggest debtor, and after each payment their amounts change. A heap gives the largest element quickly and still works when we push the leftover amount back in. Sorting the list again after every payment would be slower.

6.3 Singly Linked List

Definition. A linked list is a chain of nodes. Each node stores data and a pointer to the next node. Its size can grow or shrink while the program runs.

Where we used it. Each expense is stored as a node with its description, amount, payer and the people sharing it.

Why. We cannot know how many expenses a trip will have. A linked list grows as needed, and adding or removing the latest expense is only a pointer change, with no shifting of elements like in an array.

6.4 Stack

Definition. A stack works on the last in, first out rule. The most recent element is the first to be removed, like a pile of plates.

Where we used it. The expense list works as a stack. The newest expense is at the head, and undo removes it and reverses its effect on the balances and the graph.

Why. Undo is naturally last in, first out, since the most recent entry is the first one you want to take back. Push and pop both take O(1) time.

7. Settlement Algorithm

The whole program depends on one number for each person, the net balance:

net balance = money the person paid - the share they owe

A positive balance means the person should get money back, and a negative balance means the person has to pay. The balances of everyone always add up to zero, so this also works as a quick check that nothing went wrong.

As an example, take six friends on a Goa trip. Their net balances after all the expenses are:

Person	Net balance (Rs)	Status
Arun	+4850	gets back
Esha	+350	gets back
Bala	-250	owes
Deepa	-650	owes
Farhan	-1850	owes
Charu	-2450	owes

The algorithm works like this:

1. Put every creditor into the creditor max-heap and every debtor into the debtor min-heap.
2. Take the biggest creditor and the biggest debtor out of their heaps.
3. The debtor pays the smaller of the two amounts to the creditor.
4. Whoever still has something left goes back into their heap.
5. Repeat until both heaps are empty.

For the example above it gives:

1. Charu pays Arun Rs 2450
2. Farhan pays Arun Rs 1850
3. Deepa pays Arun Rs 550
4. Bala pays Esha Rs 250
5. Deepa pays Esha Rs 100

That is 5 payments instead of around 20 separate ones.

Every step fully settles at least one person, so a group of n people needs at most n - 1 payments. The result is usually very close to the minimum, but we do not claim it is always the absolute minimum, because finding that is a much harder problem.

If an amount does not divide evenly, for example Rs 100 among 3 people, the first few people pay Rs 1 extra. This way no rupee is lost and the balances still add up to zero.

8. System Design

Main structures in C:

- `Person`: name and net balance.
- `Expense`: description, amount, payer, the people sharing it, and a pointer to the next (older) expense.
- `Heap`: an array of person indexes and the current size. Both heaps use this structure and compare the balances of the people they hold.
- Debt matrix: a 2D array holding the raw debts between every pair of people.

Menu:

- Add a member.
- See all members.
- Add an expense.
- See who is ahead or behind.
- See who owes whom.
- Undo the last expense.
- Settle up.
- Save trip.
- Load saved trip.
- Exit.

Main functions: addMember, showMembers, addExpense, showBalances, showDebts, undoLastExpense, buildHeaps, heapInsert, heapExtract, debtorHeapInsert, debtorHeapExtract, settle, saveData, loadData.

9. Implementation Details

The whole project is written in C. Money is stored as whole rupees in integers, because floating-point numbers can give small rounding errors when an amount is split several ways. The program is menu driven and supports up to 20 members. A trip is saved to a plain text file called splitsmart.txt that has the trip name, the members with their balances, the debt matrix, and every expense with its description, amount, payer and which people share it. It is saved when the user chooses Save or Exit, and loaded when the program starts or when the user chooses Load, so the trip is still there next time. We tested it with a demo trip of 6 friends and 5 expenses, and we also checked undo and save and load.

When settling up, the program prints every payment step and the total number of payments, so it is easy to follow how the algorithm works.

10. Complexity Analysis

Operation	Structure	Time
Add an expense	Linked list (push) + balance and graph update	O(n)
Undo the last expense	Stack (pop) + reverse update	O(n)
Build the two heaps	Heap	O(n log n)
Settle up (at most n - 1 steps)	Heap	O(n log n)
Show all raw debts	Graph (matrix)	$O(n^2)$
Check one debt	Graph (matrix)	O(1)

Here n is the number of members. If we had sorted the creditors and debtors again after every payment instead of using heaps, settling up would cost $O(n^2 \log n)$. With heaps it stays at $O(n \log n)$, and the graph uses $O(n^2)$ space, which is fine for groups of this size.

11. Limitations and Future Scope

Limitations.

- Expenses can only be split equally among the chosen people. Custom amounts or percentages are not supported yet.
- The method guarantees at most n - 1 payments, but not always the absolute minimum.
- Only 20 members are supported, and an adjacency matrix wastes space for large groups.
- Amounts are whole rupees only, and the program does not actually send any money.
- The saved file is plain text, so anyone who can open it can read or change it.

Future scope.

- Add custom and percentage splits.
- Connect the final payment list to UPI so that each line opens a payment directly.
- Reduce the payments further by finding smaller groups whose balances add up to zero and settling them separately.
- Build a mobile or web front end that keeps many groups in one place.
- Handle recurring expenses, such as monthly rent and bills for flatmates.

12. Conclusion

We started with something most of us have seen, a group trip that ends in a confusing pile of small debts, and turned it into a working program. Each structure we learned in class got a real job: the graph shows the tangle of raw debts, the linked list and stack store the expenses and make undo possible, and the two heaps pick the right pair of people to settle next. The project showed us that choosing the right data structure matters as much as writing the code, and that even a small everyday problem can be solved properly when the data is organized well.